RHU LabChain Peer and ledger command guide

Prepared for your existing VPS and source tree. Verified on 28 October 2026, approximately 2:22 PM Singapore time. Commands use Bash on the server, not the browser console.

What this guide covers

Page	Task
2	Set up the read-only helper; prove Hyperledger Fabric is running
3	Stop and restore peer4 for a controlled demonstration
4	Read one peer’s ledger status and locate its persistent volume
5	Find an event reference and read its blockchain fingerprint
6	Compare the PDF fingerprint with the event commitment
7	Read a block or transaction directly from a selected peer
8	Verify active organizations, MSPs and endorsement policy
9–10	Locate the network, hashing, chaincode and verification code
11	Run a short defense demonstration and resolve common errors

Verified live snapshot

Fabric peer binary: v2.5.16. Channel: labchain-channel. Chaincode: labchain-anchor, version 1.0.0, sequence 1. All four peers returned height 17 and matching latest block hashes. The active configuration contains Org1MSP, Org2MSP and OrdererMSP, with etcdraft (Raft) ordering.

This is a snapshot, not a permanent expected height. All peers share one VPS. Matching heights alone do not prove a particular report is authentic.

What was actually executed

Only inspection and read-only queries were run to prepare this guide. The block, anchor, history and transaction commands were checked against existing ledger data. No peer was stopped or started, no report was changed, and no ledger transaction was submitted.

Only the clearly marked STOP and START commands on page 3 change a running service. Ordinary queries use the existing operator identities; they do not require opening public Fabric ports.

2 / Setup and proof of Fabric

Use the existing Fabric operator account (rhuadmin) on this VPS. It already needs Docker access and the existing organization administrator files. Do not copy private keys into the presentation or terminal output.

Initialize the short command once per terminal

```
cd /opt/rhu-labchain
rhu_peer() {
  bash /opt/rhu-labchain/docs/defense-commands/peer-read.sh "$@"
}
bash docs/defense-commands/peer-read.sh --help
```

rhu_peer is a convenience function created for this guide, not a built-in Fabric command. The helper permits only listed read operations. It selects the existing peer address, TLS certificate and organization identity. Its complete source is in the path above.

Show the real binary and deployed network

```
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
docker exec labchain-peer4 peer version
rhu_peer peer4 channels
rhu_peer peer4 definition
```

Expected: Fabric v2.5.16 inside the running peer; channel labchain-channel; chaincode labchain-anchor; approvals for Org1MSP and Org2MSP. The version command inside the container proves more than a local CLI installation alone.

Peer / container	Organization	Host TLS endpoint
peer1 / labchain-peer1	Org1MSP	127.0.0.1:7051
peer2 / labchain-peer2	Org1MSP	127.0.0.1:8051
peer3 / labchain-peer3	Org2MSP	127.0.0.1:9051
peer4 / labchain-peer4	Org2MSP	127.0.0.1:10051

Orderer: labchain-orderer / OrdererMSP / 127.0.0.1:7050. External chaincode services: labchain-anchor1 through labchain-anchor4. A chaincode service is not an additional peer.

3 / Stop and restore one peer

Use peer4 for the demonstration. This temporarily takes its Fabric endpoint offline; it does not revoke membership, remove the organization or erase its ledger. The other peers and the orderer remain running.

Before the demonstration — READ ONLY

```
rhu_peer peer4 info
docker inspect --format '{{.State.Status}}' labchain-peer4
```

STOP peer4 — CHANGES SERVICE AVAILABILITY

```
docker stop --timeout 30 labchain-peer4
docker inspect --format '{{.State.Status}}' labchain-peer4
```

Expected container state: exited. Refresh the website’s Network tab and the chosen report’s verification page. If the other three peers provide complete matching evidence and there are no other failures or pending lifecycle events, expect PARTIAL / 3 of 4, with peer4 offline. Allow the request to refresh; an old screen is not a new observation.

START the same existing peer — RESTORES SERVICE

```
docker start labchain-peer4
docker inspect --format '{{.State.Status}}' labchain-peer4
rhu_peer peer4 info
```

Wait for readiness and repeat the final query if startup is still in progress. Then refresh report verification. Full verification requires all required evidence to be fresh and matching; “running” alone is insufficient.

Check recovery if needed

```
docker logs --tail 60 labchain-peer4
cd /opt/rhu-labchain/blockchain/network
scripts/single-vps-status.sh peer4 --channel
```

Stopping only labchain-peer4 leaves its separate labchain-anchor4 service in place. If that service was independently stopped, diagnose it before restarting it. Do not use down -v, volume deletion, identity regeneration or a new channel to recover from this demonstration.

An offline peer cannot answer queries. Start that same peer to query its retained ledger, or select another running peer to query that peer’s own copy. Stopping peer4 does not remove historical copies elsewhere.

4 / Read and locate a peer ledger

Read one selected peer — READ ONLY

```
rhu_peer peer4 channels
rhu_peer peer4 info
rhu_peer peer4 definition
```

info uses peer channel getinfo against peer4’s direct TLS endpoint. height is the number of blocks: height 17 means blocks 0–16. currentBlockHash identifies the latest block header; previousBlockHash identifies its predecessor. Neither is a laboratory report’s PDF fingerprint.

Compare all four peers

```
for p in peer1 peer2 peer3 peer4; do
  printf '\n--- %s ---\n' "$p"
  rhu_peer "$p" info
done
```

Lag or an offline endpoint can make observations differ. Equal heights are a network observation. For report-specific evidence, compare its anchor and transaction and use the report verifier.

Find peer4’s persistent storage

```
docker inspect --format \
  '{{range .Mounts}}{{println .Type .Name .Destination}}{{end}}' \
  labchain-peer4
docker volume inspect --format '{{.Mountpoint}}' \
  labchain-peer4-ledger
docker exec labchain-peer4 sh -c \
  'ls -d /var/hyperledger/production/ledgersData/*'
```

The verified ledger mount is labchain-peer4-ledger → /var/hyperledger/production. Fabric keeps its block store, LevelDB world state and history/index data beneath the ledger data tree. The volume inspection returns this host’s actual mountpoint; do not assume a Docker storage root.

Use Fabric queries to inspect records. The database files are not readable JSON. Do not edit, remove or copy live LevelDB files as an ordinary way to inspect the ledger.

Open a container shell, if you need directory inspection

```
docker exec -it labchain-peer4 sh
```

Use exit to leave the container. A container shell is not necessary for rhu_peer queries. Each peer has its own separately named ledger volume.

5 / Read a report’s blockchain anchor

Get the correct identifier from the website

Sign in with BLOCKCHAIN_EXPLORER_VIEW permission. Open /administration/blockchain → Anchors. Expand Anchor details and copy the full event reference. This is event_uuid, and becomes the ledger anchor_id. Also note the record hash, transaction ID and block number.

Do not substitute the report number, entity reference, QR token or transaction ID for the event reference. Each release, revocation or supersession event has its own anchor ID.

Read that anchor from peer4 — READ ONLY

```
read -r -p "Event reference: " ANCHOR_ID
rhu_peer peer4 exists "$ANCHOR_ID"
rhu_peer peer4 anchor "$ANCHOR_ID"
rhu_peer peer4 history "$ANCHOR_ID"
```

Ledger field	Meaning
anchor_id	The event UUID used as the lookup key
content_hash	SHA-256 commitment of the canonical event
entity_reference	Opaque report UUID, not a patient identifier
previous_hash	Linked release-event commitment when applicable
source_msp / source_node	Submitting organization and logical source
transaction_id	Transaction that created the anchor

The native query inside the helper

```
# Excerpt inside the helper; TLS/admin settings are required.
peer chaincode query -C labchain-channel -n labchain-anchor \
  -c '{"Args":["ReadAnchor","PASTE_EVENT_UUID_HERE"]}'
```

Use rhu_peer for terminal execution; this native excerpt needs the TLS/admin environment selected inside the helper. ReadAnchor reads world state; GetAnchorHistory reads that key’s history. The contract has no update/delete method. Later lifecycle events create other linked anchors, so one key’s history is not the report’s entire lifecycle.

Read the same event from another peer

```
rhu_peer peer3 anchor "$ANCHOR_ID"
```

Compare content_hash and the other event fields. A matching anchor alone is narrower than the application’s full verification, which also checks committed transactions, blocks, lifecycle coverage and freshness.

6 / Check both kinds of fingerprint

There are two related hashes. D = SHA-256(exact released PDF bytes). C = SHA-256(canonical event bytes), where the event includes D, report identity, version and lifecycle fields. The ledger content_hash is C, not D.

Calculate the PDF fingerprint on the machine holding it

```
read -r -p "Path to downloaded official PDF: " pdf_file
sha256sum -- "$pdf_file"
```

Use the exact downloaded issued PDF. Editing or re-saving it can change the bytes. Compare this digest with the stored report hash, not directly with the ledger content_hash.

View all four values in the website

In /administration/blockchain → Integrity, enter the numeric Report ID. This tab additionally requires BLOCKCHAIN_INTEGRITY_VERIFY. It shows PDF SHA-256, Stored report hash, Event record hash and Ledger content hash. The first pair should agree (D); the second pair should agree (C). This Integrity tab is separate from the report scanner's independent four-peer verification.

Optional SQL — read-only, in your approved database session

If you already have authorized database access, select the application database and run this query. Replace the placeholder with the event reference. No database password or connection secret is needed in the guide.

```
SELECT event_uuid, event_status,
       JSON_UNQUOTE(JSON_EXTRACT(
         canonical_payload, '$.artifact_sha256')) AS pdf_sha256,
       record_hash AS event_sha256,
       SHA2(canonical_payload, 256) AS recomputed_event_sha256,
       fabric_transaction_id, fabric_block_number
FROM blockchain_event
WHERE event_uuid = 'PASTE_EVENT_UUID_HERE';
```

Compare event_sha256 with recomputed_event_sha256 and peer4's content_hash. This query is documented from the schema; it was not executed against the application database while preparing the guide.

Where the report file is stored

The application uses REPORT_STORAGE_DIR plus lab_report.pdf_path. The example configuration uses /var/lib/rhu-labchain/reports; the configured value is authoritative. Relative released files follow released/<year>/<report-id>-<random>.pdf. Prefer downloading through the authenticated application instead of exposing the private storage directory.

A digest match verifies matching bytes under the checks. It does not establish clinical accuracy. A QR identifies the original record but cannot authenticate altered surrounding printed values.

7 / Read blocks and transactions

These operations query QSCC on the selected peer. They do not use the orderer as a substitute for that peer’s ledger.

Export a block from peer4 — READ ONLY

```
umask 077
LEDGER_VIEW=$(mktemp -d /tmp/labchain-view.XXXXXX)
rhu_peer peer4 block 0 > "$LEDGER_VIEW/block0.json"
python3 -m json.tool "$LEDGER_VIEW/block0.json" | less
```

Block 0 is the initial channel configuration. Press q to leave less. For a report event, use the block number shown in Anchors; it must be less than that peer’s current height.

```
read -r -p "Report event block number: " BLOCK_NUMBER
rhu_peer peer4 block "$BLOCK_NUMBER" \
  > "$LEDGER_VIEW/report-block.json"
```

Read the report’s committed transaction

```
read -r -p "Transaction ID: " TX_ID
rhu_peer peer4 transaction "$TX_ID" \
  > "$LEDGER_VIEW/transaction.json"
python3 - "$LEDGER_VIEW/transaction.json" <<'PY'
import json, sys
tx = json.load(open(sys.argv[1]))
print("Validation code:", tx["validationCode"])
PY
```

Validation code 0 means VALID. A transaction can exist in a block without being valid, so mere existence is insufficient. The read helper decodes protos.ProcessedTransaction for this query.

Native peer-block query used by the helper

```
peer chaincode query -C labchain-channel -n qscc \
  -c '{"Args":["GetBlockByNumber","labchain-channel","0"]}' \
  --hex
```

This native excerpt runs inside the helper’s TLS/admin environment. The helper converts hex to exact bytes, then uses configtxlator proto_decode --type common.Block. This avoids the binary-decode failure observed with this CLI’s --raw output. Transactions use GetTransactionByID and protos.ProcessedTransaction.

Files are created only in the new local directory printed by: printf "%s\n" "$LEDGER_VIEW". Decoded ledger exports contain identifiers and public certificate metadata; use a synthetic example for public demonstration.

8 / Verify organizations and policies

Inspect the active channel configuration — READ ONLY

```
rhu_peer peer4 organizations
rhu_peer peer4 definition
```

organizations fetches the current configuration block from the orderer using the selected organization’s existing admin identity. It is explicitly an orderer configuration read, unlike the direct-peer block query on page 7. It decodes the current block and prints only a summary.

Verified configuration item	Value at inspection
Application organization / MSP	Org1 → Org1MSP; Org2 → Org2MSP
Orderer organization / MSP	OrdererOrg → OrdererMSP
Consensus type	etcdraft (Raft)
Application endorsement policy	2-of-2 peer principals: Org1MSP and Org2MSP
Committed chaincode	labchain-anchor; v1.0.0; sequence 1
Chaincode organization approvals	Org1MSP: true; Org2MSP: true

MSP means Membership Service Provider: the identity and trust rules for an organization. An organization name and its MSP ID are related but different fields. Two organizations here are logical groups on one host, not two independently operated hospitals.

Show the running peer’s own configured organization

```
docker exec labchain-peer4 sh -c \
  'printf "%s %s\n" "$CORE_PEER_ID" "$CORE_PEER_LOCALMSPID"'
# Expected: peer4 Org2MSP
```

Where these values come from

The active configuration read is stronger evidence than a source-file claim. The source declares the intended organizations and policy in blockchain/network/config/configtx.single-vps.yaml. Peer-specific MSP IDs are also declared in compose/docker-compose.single-vps.yml.

Endorsement is a transaction acceptance rule. The application’s four-peer verification is a later report-evidence rule. They are not the same vote. Four matching observations are required for a full report verification result; contradictory evidence cannot be overridden by a majority.

There is no running Fabric CA service in this prototype configuration: identities were bootstrapped with cryptogen. Absence of a Fabric CA container does not mean the peers are not Hyperledger Fabric.

9 / Network and chaincode code locations

All paths below are relative to /opt/rhu-labchain. Line numbers refer to the source inspected for this guide; they may move after edits.

File : starting line	What to show during defense
blockchain/network/compose/ docker-compose.single-vps.yml:62	peer1 service. peer2 starts at 167; peer3 at 272; peer4 at 377. Images, MSP IDs, TLS, ports and ledger volumes.
blockchain/network/config/ configtx.single-vps.yaml:2	Org1, Org2 and OrdererOrg. Endorsement policy at 43; Raft configuration at 46.
blockchain/network/versions.json:1	Pinned Fabric version and reviewed image digests; cryptogen bootstrap and no CA service.
blockchain/network/scripts/ single-vps-common.sh:45	single_peer(): maps peer number to organization, host port, TLS root and admin identity.
blockchain/network/scripts/ single-vps-status.sh:1	Read-only health, joined channels and optional ledger info.
blockchain/chaincode/labchain-anchor/ lib/anchor-contract.js:30	CreateAnchor validates identity and fields; rejects an existing key; stores content_hash at line 47 and writes at line 55.
blockchain/chaincode/labchain-anchor/ lib/anchor-contract.js:60	ReadAnchor reads world state. AnchorExists starts at 67; GetAnchorHistory at 72.
docs/defense-commands/peer-read.sh:1	New read-only command helper supplied with this guide. No invoke or service-control operation is allowed.

Print the exact source with line numbers

```
cd /opt/rhu-labchain
nl -ba blockchain/network/config/configtx.single-vps.yaml \
  | sed -n '1,80p'
nl -ba blockchain/chaincode/labchain-anchor/lib/anchor-contract.js \
  | sed -n '27,93p'
```

To locate a function after edits, use rg -n with its name instead of relying only on an old line number.

10 / Application and fingerprint code

File : starting line	Responsibility
app/services/report_release_service.py:119	hashlib.sha256(data).hexdigest() calculates D from exact PDF bytes. Line 120 stores report_hash; line 130 captures the release event.
app/services/ blockchain_event_validation.py:38	canonicalize(): fixed event fields and serialization. Line 63 uses sorted keys and compact JSON.
app/services/ blockchain_outbox_service.py:25	capture(): builds event. Line 56 calculates C as SHA-256 of UTF-8 canonical_payload.
app/services/ blockchain_delivery_service.py:207	Worker delivers or reconciles an anchor. confirm() at 191 checks event fields before confirmation.
blockchain/gateway-adapter/src/adapter.js:25	Proposal, endorsement, submission and commit-status checks. This is the write integration.
blockchain/gateway-adapter/src/monitor.js:13	Peer/organization endpoints. openPeer at 109; directSession at 121; committedReport at 260; reportEvidence at 289.
app/services/public_authenticity_service.py:38	aggregate_peer_evidence() checks four-peer agreement. ledger_evidence at 103; public_verification at 141.
app/services/ blockchain_monitor_service.py:82	Integrity comparison of PDF, stored hash, immutable event and live ledger evidence.
frontend/src/pages/BlockchainMonitor.tsx:110	Anchor details show event reference and record hash; line 121 shows both pairs of integrity hashes.
app/api/blockchain_monitor.py:14	Permission-protected explorer routes. Integrity requires an additional permission.

Find the fingerprint calculations quickly

```
cd /opt/rhu-labchain
rg -n 'sha256|capture_release' \
  app/services/report_release_service.py
rg -n 'canonical_payload|record_hash|sha256' \
  app/services/blockchain_outbox_service.py
rg -n 'content_hash|putState|ReadAnchor' \
  blockchain/chaincode/labchain-anchor/lib/anchor-contract.js
```

11 / Defense sequence and troubleshooting

A short demonstration sequence

1. Show the running container’s Fabric version.
2. Show organizations, channel and committed chaincode.
3. Select a synthetic released report and copy its event reference.
4. Read its anchor from peer4; explain content_hash.
5. Show the PDF hash pair and event-hash pair in Integrity.
6. Optionally stop peer4 using page 3; refresh report verification.
7. Restart peer4, confirm readiness and refresh verification again.
8. Show the hashing and CreateAnchor source locations.

Common problems

Symptom	Meaning / next check
rhu_peer: command not found	Repeat the function definition on page 2 in this terminal.
Missing tool / Python YAML error	Use the existing operator environment and pinned tools. Default tool path: blockchain/network/tools. Do not replace network identities to fix tool errors.
Connection refused / unavailable	The selected peer may be stopped or not ready. Check its state and logs; repeat after readiness.
Anchor not found	Check event reference, target peer and channel; confirm delivery is CONFIRMED. A report UUID or QR token is not an anchor ID.
Height matches but report not verified	Inspect that report’s commitment, transaction validity and lifecycle coverage. Height is not report-specific proof.
Permission denied in the website	Explorer needs BLOCKCHAIN_EXPLORER_VIEW; Integrity additionally needs BLOCKCHAIN_INTEGRITY_VERIFY. Use the approved account.
Three peers match, but result is CONFLICT	Contradictory evidence takes precedence; do not treat three votes as full verification.

Validation record and limits

Checked: Bash syntax, pinned CLI and running peer binary, all four ledger heights, channel membership, committed definition, active organization/policy summary, peer4 block decoding, existing anchor/history reads, and a committed transaction with validation code 0. No new transaction was submitted. Stop/start instructions and optional SQL were not executed for this guide.

Sources: current local source files listed on pages 9–10; blockchain/docs/OPERATIONS.md; docs/FOUR_PEER_QR_VERIFICATION_COMPLETION.md; docs/PHASE_8B_TRANSACTIONAL_OUTBOX.md. Earlier Phase 8A documents describe an earlier implementation stage; the live snapshot here was checked separately.